

ConsultHours

Registro de horas facturables para una consultora
Billable hours tracking for a consultancy

Documentación completa del proyecto
Complete Project Documentation

Ejercicio técnico / Technical exercise

Autor / Author: Uriel Soto

Fecha / Date: 1 de septiembre de 2026 / September 1st, 2026

Stack: PHP 8 · MySQL (XAMPP) · JavaScript · HTML/CSS · Python 3 · Git

Parte I: Español (páginas siguientes) · Part II: English (segunda mitad del documento)

Índice — Parte I (Español)

1. Resumen del proyecto
2. Instalación y ejecución
3. Estructura del proyecto

- 4. Arquitectura técnica
- 5. Base de datos: cómo se usó
- 6. Seguridad: riesgos identificados y mitigaciones
- 7. Decisiones de negocio (las dos que el ejercicio deja abiertas)
- 8. Lenguajes y herramientas: qué se usó y para qué
- 9. Uso de IA (Claude Code / Claude): qué se pidió, qué decisiones se tomaron, qué se corrigió
- 10. Casos reales: errores encontrados y cómo se corrigieron
- 11. Pruebas y verificación
- 12. Git y GitHub: cómo se usaron y qué se aprendió
- 13. Aprendizajes finales y checklist de entrevista

Part II (English) sigue después de la Parte I, con las mismas secciones. / Part II (English) follows Part I, with the same sections.

Parte I — Español

1. Resumen del proyecto

ConsultHours es un registro de horas facturables para una consultora: los consultores capturan las horas que trabajaron para cada cliente, y el sistema produce un resumen mensual de horas facturables por cliente, con control de acceso en dos niveles (autenticación y autorización por rol/dueño).

Decisión de partida importante: el enunciado venía acompañado de un repositorio inicial en Node.js + SQLite con vulnerabilidades sembradas para corregir. **Se decidió no usarlo y construir el sistema completo desde cero**, en un stack distinto elegido a propósito (PHP + MySQL vía XAMPP + JavaScript vanilla), porque el entrevistador dejó claro que no quería una solución copiada y pegada, sino evidencia de que cada decisión se entiende y se puede defender. Cada línea de este proyecto se escribió a propósito.

Capa	Tecnología
Backend	PHP 8 (PDO + MySQL), sesiones nativas de PHP, sin framework
Base de datos	MySQL (vía XAMPP), 4 tablas, integridad referencial con llaves foráneas

Frontend	HTML5 + CSS3 (animaciones 3D reales, GPU) + JavaScript vanilla, sin build step
Verificación	Python 3 (2 scripts: verificación independiente del resumen + suite de 15 pruebas del API)
Automatización	check_all.sh / check_all.ps1 — un comando que corre todas las verificaciones
Control de versiones	Git (commits incrementales) + GitHub para la entrega

2. Instalación y ejecución

1. **Requisitos:** XAMPP (Apache + MySQL + PHP 8) y Python 3. El proyecto vive en `C:\xampp\htdocs\PRUEBA TECNICA`.
2. **Crear y poblar la base de datos** — dos opciones:
 - **Opción A (phpMyAdmin, recomendada):** abrir `http://localhost/phpmyadmin`, pestaña SQL, pegar el contenido completo de `database/seed.sql` y dar Continuar. Crea la base, las tablas y los datos de prueba en un solo paso. Es seguro re-ejecutarlo: siempre empieza borrando la base si ya existía.
 - **Opción B (línea de comandos):** `mysql -u root < database/schema.sql` y luego `php database/seed.php`.
3. **Abrir el programa:** `http://localhost/PRUEBA%20TECNICA/` (redirige automáticamente al frontend).
4. **Usuarios de prueba** (en la pantalla de login son clicables y autocompletan el formulario):

Usuario	Contraseña	Rol
admin	admin123	Administrador
carla	carla2024	Consultor
miguel	miguel!!	Consultor

3. Estructura del proyecto

PRUEBA TECNICA/	
— .htaccess	# Hace que "/" sirva public/index.php
— public/index.php	# Punto de entrada (redirige al frontend)
— database/	# Protegida por .htaccess (no accesible por URL)
— schema.sql	# Solo el esquema (tablas vacías)
— seed.sql	# Esquema + datos, listo para phpMyAdmin
— seed_data.json	# Datos de prueba (fuente única de verdad)
— seed.php	# Puebla MySQL desde seed_data.json (CLI)
— backend/	
— config/database.php	# Conexión PDO (charset=utf8mb4 fijo en el DSN)
— includes/	# auth.php, functions.php, bootstrap.php
— api/	# login, logout, session, clients, consultants, # records (GET/POST/DELETE), summary
— frontend/	
— .htaccess	# Cache-Control: no-cache para .js/.css
— index.html	# SPA de una sola página
— assets/css + assets/js	# style, animations / api, auth, records, summary, main
— scripts/	# Protegida por .htaccess
— verify_summary.py	# Verificación independiente del resumen
— test_api.py	# Suite de 15 pruebas del API (unittest)
— check_all.sh / .ps1	# Todas las verificaciones en un comando
— docs/	# DOCUMENTACION.md, GIT.md, APRENDIZAJES.md
— NOTES.md	# Lo que pide el ejercicio: seguridad, decisiones, IA
— README.md	# Instalación rápida

public/ es la única carpeta pensada para abrirse por URL; database/, scripts/, backend/config/ y backend/includes/ tienen su propio .htaccess con Require all denied porque contienen cosas que jamás deben descargarse por el navegador (contraseñas de prueba, esquema, código fuente de Python).

4. Arquitectura técnica

Todo se sirve desde el **mismo origen** ([http://localhost/PRUEBA TECNICA/...](http://localhost/PRUEBA_TECNICA/...)): el frontend llama al backend con rutas relativas (../backend/api/...), así que no hace falta configurar CORS en absoluto — la opción más segura fue también la más simple.

- **Backend sin framework:** cada endpoint es un archivo PHP plano bajo backend/api/; la lógica compartida (autenticación, respuestas JSON, manejo de errores) vive en backend/includes/ y cada endpoint la carga con un solo require_once bootstrap.php.
- **Frontend SPA vanilla:** una sola página con dos pestañas (Registros / Resumen mensual) que se alternan con clases CSS; api.js centraliza todas las llamadas HTTP (cookies de sesión, header anti-CSRF, manejo de errores) en una sola función.
- **El punto de entrada** (public/index.php) calcula la ruta de redirección desde SCRIPT_NAME (no a mano), para funcionar igual sin importar si se llega por la raíz del proyecto (vía DirectoryIndex) o visitando la URL directa.

- **Refresco de datos:** cada cambio de pestaña vuelve a pedir los datos del panel que se muestra, y crear/borrar un registro también refresca el resumen — así ninguna pantalla se queda con números viejos.

Endpoints del API

Método	Ruta	Qué hace	Quién puede
POST	/api/auth/login.php	Inicia sesión (bcrypt + bloqueo por intentos)	Credenciales válidas
POST	/api/auth/logout.php	Cierra sesión	Sesión iniciada
GET	/api/auth/session.php	Usuario de la sesión actual	Sesión iniciada
GET	/api/clients.php	Catálogo de clientes	Sesión iniciada
GET	/api/consultants.php	Lista de consultores (para filtros)	Solo admin
GET	/api/records.php	Lista/busca registros (q, cliente, mes, consultor)	Cada quien lo suyo; admin todo
POST	/api/records.php	Crea registro (dueño = sesión, SIEMPRE)	Sesión iniciada
DELETE	/api/records.php?id=	Borra registro	Dueño o admin
GET	/api/summary.php	Resumen mensual facturable por cliente	Cada quien lo suyo; admin agregado o filtrado

5. Base de datos: cómo se usó

Cuatro tablas en MySQL (`database/schema.sql`), todas InnoDB/utf8mb4:

Tabla	Propósito	Decisión de diseño relevante
users	Consultores y administradores	Contraseñas solo como hash bcrypt (password_hash), nunca texto plano; rol como ENUM('admin','consultant')
clients	Clientes a los que se facturan horas	Nombre UNIQUE
time_records	Horas trabajadas	hours es derivado (se calcula de start/end en el servidor, nunca se confía en un número suelto del cliente); llaves foráneas reales a users y clients; índice (consultant_id, work_date) porque la detección de traslapes y el resumen filtran por esas columnas
login_attempts	Intentos de login fallidos	Sin FK a users a propósito: un intento con un usuario inexistente también cuenta para el bloqueo (evita enumerar qué usuarios existen)

Tres caminos para poblar los datos de prueba, una sola fuente de verdad: `seed_data.json` alimenta tanto a `seed.php` (CLI) como a `verify_summary.py` (el cálculo esperado), para que no puedan desincronizarse. `seed.sql` (phpMyAdmin) se genera de esos mismos datos.

Lección aprendida a la mala: `seed.sql` empieza con `SET NAMES utf8mb4`; porque sin eso, el cliente de línea de comandos de MySQL en Windows puede usar la página de códigos de la consola en vez de UTF-8 y guardar los acentos corruptos de forma permanente (pasó de verdad — ver sección 10, caso 4).

6. Seguridad: riesgos identificados y mitigaciones

Como el sistema se construyó desde cero, los riesgos se identificaron **durante el diseño** (no como un diff sobre código ajeno). Los doce, con su mitigación:

#	Riesgo	Mitigación implementada
---	--------	-------------------------

1	Suplantación del dueño al crear un registro (el bug explícito del enunciado)	consultant_id se toma SIEMPRE de la sesión del servidor; el body del cliente jamás se lee para ese campo. Probado enviando el id de otro consultor: se ignora.
2	Confundir autenticación con autorización	Dos capas separadas y explícitas: require_auth() (¿hay sesión?) vs. require_admin()/can_manage_record() (¿puede esto sobre este recurso?)
3	Inyección SQL	PDO con sentencias preparadas reales (ATTR_EMULATE_PREPARES=false) en el 100% de las consultas, incluida la búsqueda con LIKE
4	Contraseñas en texto plano	password_hash()/password_verify() (bcrypt); verificación contra hash dummy cuando el usuario no existe (anti-timing)
5	Fijación de sesión	session_regenerate_id(true) tras cada login exitoso
6	CSRF (API basada en cookies)	Header personalizado X-Requested-With obligatorio en POST/DELETE + cookie SameSite=Lax + HttpOnly
7	Fuerza bruta en login	Bloqueo por USUARIO persistido en BD (5 intentos / 5 min) — borrar cookies ya no reinicia el contador; todo el cálculo de tiempo con el reloj de MySQL (ver sección 10, caso 2)
8	Fuga de detalles internos en errores	display_errors=0 + manejadores globales que responden mensaje genérico y registran el detalle solo en el log del servidor
9	XSS al mostrar datos	escapeHtml() en el frontend antes de insertar cualquier texto en el DOM (defensa en profundidad)
10	Entrada inválida	Validación server-side de fecha, horario (inicio < fin), rango de horas (0–24), cliente existente, longitud de descripción
11	CORS mal configurado	Eliminado por diseño: mismo origen para frontend y backend, cero cabeceras CORS

12	Archivos internos servidos por Apache	.htaccess con Require all denied en database/, scripts/, backend/config e includes/ — antes, seed_data.json (contraseñas de prueba) respondía 200 a cualquiera; confirmado 403 después
----	---------------------------------------	--

7. Decisiones de negocio (las dos que el ejercicio deja abiertas)

7.1 Horas traslapadas el mismo día (ejemplo real: 6 de agosto)

Decisión: no bloquear; detectar y marcar visualmente. El registro traslapado se guarda, pero tanto la respuesta de creación como el listado lo marcan (insignia "⚠ traslape") para que el consultor o un admin lo revisen.

Por qué: un consultor puede tener razones legítimas (cambio de contexto rápido, llamada que se solapó con una reunión, corrección posterior). Bloquear convierte un problema de *datos por revisar* en un problema de *flujo de trabajo roto*; recortar horas automáticamente sería peor (el sistema decidiría en silencio cuál registro es "el bueno"). Marcar y dejar visible da la información para que un humano decida. El punto de extensión para endurecer la regla después ya existe en el código.

7.2 Visibilidad del resumen financiero entre consultores

Decisión: cada consultor ve solo su propio resumen (y solo sus propios registros); solo el administrador ve el agregado de todos o filtra por consultor.

Por qué: las horas facturables están ligadas a cuánto se cobra y al desempeño de cada quien — es información sensible como un salario, no un dato operativo neutro. El enunciado ya establece el precedente equivalente para el borrado ("solo sus propios registros"); extender el mismo criterio de dueño a la lectura es consistente. La alternativa (que todos vean todo) se descartó: no aporta al trabajo del consultor y sí crea un riesgo de privacidad sin justificación de negocio.

8. Lenguajes y herramientas: qué se usó y para qué

Lenguaje / herramienta	Rol en el proyecto	Por qué ese y no otro
PHP 8	Todo el backend: API, sesiones, conexión a MySQL	Corre nativo en el Apache de XAMPP sin build step ni proceso aparte — el requisito era "córrelo desde XAMPP"

MySQL / SQL	Almacenamiento con integridad referencial real	A este dominio le importa que un registro huérfano sea imposible (llaves foráneas), no solo "raro"
JavaScript (vanilla)	Toda la lógica del frontend: fetch, sesión, render, formularios	Una SPA de 2 pantallas no justifica framework + bundler; se abre directo en el navegador sin npm install
HTML5	Estructura semántica de la única página	—
CSS3	Layout, glassmorphism y TODAS las animaciones 3D	transform/perspective/@keyframes nativos alcanzan sin ninguna librería; solo se animan transform y opacity (GPU) para que sea fluido sin lag; prefers-reduced-motion respetado
Python 3	Verificación independiente del resumen (verify_summary.py) y suite de 15 pruebas del API (test_api.py, unittest, sin dependencias)	A propósito un lenguaje distinto al del backend: si la verificación compartiera código con summary.php, un bug podría verse bien en ambos lados. Escribir el cálculo dos veces, en dos lenguajes, hace que la coincidencia sea evidencia real
Bash / PowerShell	Durante el desarrollo: probar con curl/mysql. Como entregable: check_all.sh/.ps1 (linting + ambas suites de Python en un comando)	La automatización que une PHP + JS + Python en un solo lugar verificable, con código de salida 0/1 listo para pre-commit o CI
Apache (.htaccess)	DirectoryIndex en la raíz; Require all denied en carpetas internas; no-cache en JS/CSS	Seguridad y corrección en la capa donde vive el problema (el servidor), no parches en la aplicación
Git + GitHub	Historial incremental (37+ commits) y entrega al entrevistador	Ver sección 12

9. Uso de IA (Claude Code / Claude)

El proyecto se construyó con **Claude Code** como asistente de programación en una sesión interactiva. Esto es exactamente lo que el enunciado pide documentar: qué se le pidió, qué decisiones se tomaron con ella, y qué hubo que corregir de lo que propuso.

9.1 Qué se le pidió

- Construir el sistema completo descrito en el enunciado, desde cero, en PHP + MySQL (XAMPP) + JS con animaciones 3D, con todo el código comentado en español e inglés y organizado en carpetas.
- Antes de escribir código, la IA pidió confirmar dos ambigüedades: si debía construir el sistema o solo entregar prompts, y qué rol tendría Python. Se resolvió: construir todo, con Python como verificador independiente.
- **Probar de verdad, no solo escribir:** correr el schema y el seed contra el MySQL real, probar cada endpoint con curl (incluidos los intentos de suplantación y acceso no autorizado), correr los scripts de Python contra el API vivo, y tomar capturas de pantalla reales del navegador (vía Chrome DevTools Protocol) para confirmar visualmente la interfaz.

9.2 Decisiones tomadas trabajando con la IA

- Las dos decisiones de negocio abiertas (sección 7): la IA propuso cada una **con justificación explícita y verificable**, no como regla arbitraria — el criterio para aceptarlas no fue "la IA lo escribió" sino "la razón se sostiene si alguien la cuestiona".
- No usar el repositorio de partida (decisión de la persona, no de la IA) y el stack PHP/MySQL/JS/Python (decisión conjunta, confirmada al inicio).
- Qué NO agregar: paginación para 20 filas de demo, edición de registros que nadie pidió, tema claro — se documentó por qué agregar features sin justificación es sobre-ingeniería que resta, no suma.

9.3 Qué hubo que corregir de lo que la IA propuso (con evidencia)

1. **Contraste del gráfico de barras:** la primera versión tenía las tres barras visualmente idénticas aunque median 100%, 54% y 23%. Se detectó comparando el ancho computado del DOM contra la captura de pantalla real — la lógica estaba bien, el CSS no.
2. **Ruteo del punto de entrada:** el primer boceto de index.php hacía include del HTML (rompía todas las rutas relativas); se cambió a redirección calculada desde SCRIPT_NAME.
3. **El seed para phpMyAdmin:** funcionaba por consola y falló en el phpMyAdmin real (#1701, TRUNCATE + llaves foráneas). Ver sección 10, caso 1.
4. **El bloqueo de login no bloqueaba:** reloj de PHP y de MySQL con 8 horas de diferencia en la misma máquina. Ver sección 10, caso 2.
5. **El resumen mensual no se refrescaba solo** y la primera prueba del arreglo engañaba por caché del navegador. Ver sección 10, caso 5.

El patrón de todo el proyecto: la IA propone rápido, pero cada pieza se dio por buena solo con evidencia de ejecución real (curl, capturas, bytes en hexadecimal, tráfico de red) — nunca por "el código se ve

bien". Dos de los bugs más importantes (los acentos corruptos y el resumen sin refrescar) los encontró la persona probando la aplicación en su propio navegador, no la IA ni las pruebas automatizadas: la revisión humana final no es opcional.

10. Casos reales: errores encontrados y cómo se corrigieron

Cinco casos que pasaron de verdad durante el desarrollo — cada uno con causa raíz y corrección, no solo el síntoma (detalle completo en [docs/APRENDIZAJES.md §5](#)):

#	Síntoma	Causa raíz	Corrección
1	phpMyAdmin fallaba con #1701 al importar seed.sql (por consola funcionaba)	La casilla "revisión de claves foráneas" de phpMyAdmin ignora el SET FOREIGN_KEY_CHECKS=0 del script, y TRUNCATE falla sobre tablas referenciadas por FK aunque estén vacías	Eliminar la necesidad de TRUNCATE de raíz (el script ya parte de DROP DATABASE, las tablas nacen vacías)
2	El bloqueo por intentos de login nunca se activaba, sin error visible	PHP y MySQL con relojes ~8 horas aparte (zonas horarias configuradas por separado); la ventana calculada en PHP nunca coincidía con los timestamps de MySQL	Todo el cálculo de tiempo dentro de la consulta SQL (NOW()/INTERVAL/TIMESTAMPDIFF) — una sola fuente de verdad para la hora
3	check_all.ps1 reportaba fallo aunque las 15 pruebas pasaran	PowerShell 5.1 envuelve el stderr de un exe nativo en ErrorRecords al redirigir con 2>&1, y unittest escribe su resultado en stderr por diseño	Delegar la redirección a cmd /c y leer \$LASTEXITCODE real
4	Acentos como "└─" en la pantalla real (encontrado por la persona, no por la IA)	Los bytes UTF-8 correctos se reinterpretaban bajo CP437 al importar por consola sin forzar la codificación — corrupción permanente en la base, verificada con HEX()	SET NAMES utf8mb4 como primera instrucción de seed.sql: el archivo declara su codificación en vez de asumir la del cliente
5	El resumen mensual mostraba números viejos	Dos bugs encimados: (a) loadSummary() solo se disparaba al cambiar el valor del selector; (b) la primera prueba del arreglo engañaba porque el navegador	(a) refrescar el panel en cada cambio de pestaña y tras crear/borrar; (b)

	tras crear un registro	tenía en caché el JS de antes del cambio (Apache no mandaba Cache-Control en estáticos)	frontend/.htaccess con no-cache, must-revalidate para .js/.css
--	------------------------	---	--

11. Pruebas y verificación

- **scripts/verify_summary.py** — el enunciado pide no confiar a ciegas en el número del resumen: este script recalcula en Python, desde seed_data.json, cuántas horas facturables debería reportar cada cliente por mes y por consultor, inicia sesión de verdad contra el backend y compara. Resultado: los 17 cruces (2 meses × consultores + agregado admin) coinciden.
- **scripts/test_api.py** — 15 pruebas automatizadas (unittest, sin dependencias) que codifican todo lo probado a mano: autenticación (401 sin sesión, 401 con contraseña mala, el hash nunca sale en la respuesta), la suplantación del punto 4 del enunciado, autorización por dueño al borrar (403 ajeno / 200 propio / admin cualquiera), visibilidad de registros y resumen entre consultores, la detección de traslapes con el ejemplo del 6 de agosto, inyección SQL en la búsqueda, y el bloqueo por intentos (con un usuario inventado para no bloquear cuentas reales). Cada prueba limpia sus propios datos.
- **scripts/check_all.sh / .ps1** — un solo comando: sintaxis de cada PHP (php -l), sintaxis de cada JS (node --check), test_api.py y verify_summary.py. Sale con código 0 solo si todo pasó.

12. Git y GitHub: cómo se usaron y qué se aprendió

Git no es una función del programa: la aplicación corre igual sin la carpeta .git. Es la herramienta con la que se entrega **el proceso de construcción**, commit por commit, que es exactamente lo que pide el enunciado ("commits incrementales normales, no un solo commit final").

- **37+ commits incrementales**, cada uno con un mensaje que explica la intención (no solo el qué); cada commit compila y funciona por sí solo — no hay commits "WIP" rotos a medio camino.
- El orden cuenta la historia real: primero datos (schema/seed), luego la capa compartida de seguridad, luego endpoints, frontend, verificación en Python, documentación — y después las rondas de mejoras con sus bugs encontrados y corregidos.
- La tabla completa de cada commit, con traducción al inglés de su propósito, está en docs/GIT.md.

Git vs. GitHub: git corre local; GitHub es el servicio que hospeda el repositorio para compartirlo. Publicarlo (pasos exactos en docs/GIT.md): crear el repositorio en github.com sin README inicial, `git remote add origin ...`, `git branch -M main`, `git push -u origin main`, verificar que la pestaña Commits muestre el historial, y compartir la URL con el entrevistador (o invitarlo como colaborador si es privado). Lo que el revisor ve: el código completo, el README en la portada, y cada commit con su diff — el proceso, no solo el resultado.

13. Aprendizajes finales y checklist de entrevista

- **Autenticación y autorización son dos preguntas distintas** — y el código las separa como funciones distintas.
- **Nunca confiar en datos que controla el cliente** para decidir identidad o dueño — la sesión del servidor es la única autoridad.
- **"Funciona en mi prueba" ≠ "funciona en la herramienta real"** (phpMyAdmin, PowerShell, el navegador con caché).
- **Cuando dos sistemas deben coincidir en algo (hora, codificación), que uno solo sea el dueño** — no sincronizar dos.
- **Los errores más difíciles fallan en silencio** — el bloqueo que no bloqueaba no lanzó ninguna excepción.
- **Las pruebas automatizadas reducen el riesgo pero no reemplazan abrir la aplicación real** — dos bugs los encontró la persona, no la máquina.
- **Restraint es una feature:** saber qué NO agregar (y poder decir por qué) vale tanto como lo agregado.

Si preguntan...	La respuesta está en...
¿Por qué no usaste el repositorio que te dieron?	NOTES.md (inicio) — pedido explícito de no copiar/pegar; todo escrito desde cero
¿Cómo evitas que un consultor vea/borre lo de otro?	NOTES.md §1.1–1.2 + backend/includes/auth.php
¿Cómo sabes que el resumen es correcto?	scripts/verify_summary.py + NOTES.md §4 (evidencia real)
¿Qué pasa con horas traslapadas?	NOTES.md §2.1 — decisión justificada
¿Por qué Python si el backend es PHP?	Sección 8 — verificación independiente real
Cuéntame de un bug real que resolviste	Sección 10 — cinco, con causa raíz
¿Tienes pruebas automatizadas?	test_api.py (15) + check_all — sección 11

Part II — English

1. Project overview

ConsultHours is a billable-hours tracker for a consultancy: consultants log the hours they worked for each client, and the system produces a monthly summary of billable hours per client, with two-level access control (authentication and role/owner authorization).

Key starting decision: the exercise came with a starter repository in Node.js + SQLite with seeded vulnerabilities to fix. **I chose not to use it and to build the entire system from scratch**, in a deliberately different stack (PHP + MySQL via XAMPP + vanilla JavaScript), because the interviewer made it clear they didn't want a copy-paste solution but evidence that every decision is understood and defensible. Every line in this project was written on purpose.

Layer	Technology
Backend	PHP 8 (PDO + MySQL), native PHP sessions, no framework
Database	MySQL (via XAMPP), 4 tables, real referential integrity via foreign keys
Frontend	HTML5 + CSS3 (real 3D animations, GPU) + vanilla JavaScript, no build step
Verification	Python 3 (2 scripts: independent summary verification + a 15-test API suite)
Automation	check_all.sh / check_all.ps1 — one command that runs every check
Version control	Git (incremental commits) + GitHub for delivery

2. Installation and running

1. **Requirements:** XAMPP (Apache + MySQL + PHP 8) and Python 3. The project lives at `C:\xampp\htdocs\PRUEBA TECNICA`.
2. **Create and seed the database** — two options:
 - **Option A (phpMyAdmin, recommended):** open `http://localhost/phpmyadmin`, SQL tab, paste the full contents of `database/seed.sql` and hit Go. Creates the database, tables and test data in one step. Safe to re-run: it always starts by dropping the database if it existed.
 - **Option B (command line):** `mysql -u root < database/schema.sql` then `php database/seed.php`.
3. **Open the app:** `http://localhost/PRUEBA%20TECNICA/` (redirects automatically to the frontend).
4. **Test users** (clickable on the login screen — they autofill the form):

Username	Password	Role
admin	admin123	Administrator
carla	carla2024	Consultant
miguel	miguel!!	Consultant

3. Project structure

```
PRUEBA TECNICA/
├── .htaccess                # Makes "/" serve public/index.php
├── public/index.php         # Entry point (redirects to the frontend)
├── database/               # Protected by .htaccess (not reachable by URL)
│   ├── schema.sql          # Schema only (empty tables)
│   ├── seed.sql            # Schema + data, phpMyAdmin-ready
│   ├── seed_data.json      # Test data (single source of truth)
│   └── seed.php            # Seeds MySQL from seed_data.json (CLI)
├── backend/
│   ├── config/database.php # PDO connection (charset=utf8mb4 fixed in DSN)
│   ├── includes/          # auth.php, functions.php, bootstrap.php
│   └── api/               # login, logout, session, clients, consultants,
│                           # records (GET/POST/DELETE), summary
├── frontend/
│   ├── .htaccess          # Cache-Control: no-cache for .js/.css
│   └── index.html         # Single-page app
```

```

├── assets/css + assets/js # style, animations / api, auth, records, summary, main
├── scripts/               # Protected by .htaccess
│   ├── verify_summary.py  # Independent summary verification
│   ├── test_api.py        # 15-test API suite (unittest)
│   └── check_all.sh / .ps1 # Every check in one command
├── docs/                  # DOCUMENTACION.md, GIT.md, APRENDIZAJES.md
├── NOTES.md               # What the exercise requires: security, decisions, AI
└── README.md              # Quick start

```

`public/` is the only folder meant to be opened by URL; `database/`, `scripts/`, `backend/config/` and `backend/includes/` each carry a `.htaccess` with `Require all denied` because they hold things that must never be downloadable through the browser (test passwords, the schema, Python source).

4. Technical architecture

Everything is served from the **same origin** ([http://localhost/PRUEBA TECNICA/...](http://localhost/PRUEBA_TECNICA/)): the frontend calls the backend with relative paths (`../backend/api/...`), so no CORS configuration is needed at all — the safest option was also the simplest.

- **Framework-less backend:** each endpoint is a plain PHP file under `backend/api/`; shared logic (auth, JSON responses, error handling) lives in `backend/includes/` and each endpoint loads it with a single `require_once bootstrap.php`.
- **Vanilla SPA frontend:** one page with two tabs (Records / Monthly summary) toggled via CSS classes; `api.js` centralizes every HTTP call (session cookies, anti-CSRF header, error handling) in one function.
- **The entry point** (`public/index.php`) computes its redirect target from `SCRIPT_NAME` (not hardcoded), so it behaves identically whether reached through the project root (via `DirectoryIndex`) or by direct URL.
- **Data freshness:** every tab switch re-fetches the visible panel's data, and creating/deleting a record also refreshes the summary — so no screen is ever left showing stale numbers.

API endpoints

Method	Path	What it does	Who may
POST	<code>/api/auth/login.php</code>	Logs in (bcrypt + attempt lockout)	Valid credentials
POST	<code>/api/auth/logout.php</code>	Logs out	Logged-in session

GET	/api/auth/session.php	Current session's user	Logged-in session
GET	/api/clients.php	Client catalog	Logged-in session
GET	/api/consultants.php	Consultant list (for filters)	Admin only
GET	/api/records.php	Lists/searches records (q, client, month, consultant)	Own records only; admin sees all
POST	/api/records.php	Creates a record (owner = session, ALWAYS)	Logged-in session
DELETE	/api/records.php?id=	Deletes a record	Owner or admin
GET	/api/summary.php	Monthly billable summary per client	Own only; admin aggregate or filtered

5. Database: how it was used

Four MySQL tables (database/schema.sql), all InnoDB/utf8mb4:

Table	Purpose	Relevant design decision
users	Consultants and administrators	Passwords stored only as bcrypt hashes (password_hash), never plain text; role as ENUM('admin','consultant')
clients	Clients that hours are billed to	UNIQUE name
time_records	Hours worked	hours is derived (computed from start/end on the server — a loose client-sent number is never trusted); real foreign keys to users and

		clients; index on (consultant_id, work_date) because overlap detection and the summary filter on exactly those columns
login_attempts	Failed login attempts	Deliberately no FK to users: an attempt against a nonexistent username also counts toward the lockout (prevents user enumeration)

Three seeding paths, one source of truth: `seed_data.json` feeds both `seed.php` (CLI) and `verify_summary.py` (the expected numbers), so they can never drift apart. `seed.sql` (phpMyAdmin) is generated from the same data.

Lesson learned the hard way: `seed.sql` opens with `SET NAMES utf8mb4`; because without it, the MySQL command-line client on Windows can pick up the console's code page instead of UTF-8 and permanently store corrupted accented characters (it genuinely happened — see section 10, case 4).

6. Security: identified risks and mitigations

Since the system was built from scratch, the risks were identified **during design** (not as a diff over someone else's code). All twelve, with their mitigation:

#	Risk	Implemented mitigation
1	Owner impersonation when creating a record (the exercise's explicit bug)	consultant_id is ALWAYS taken from the server session; the client body is never read for that field. Tested by sending another consultant's id: ignored.
2	Confusing authentication with authorization	Two explicit, separate layers: <code>require_auth()</code> (is there a session?) vs. <code>require_admin()/can_manage_record()</code> (may THIS act on THIS resource?)
3	SQL injection	PDO with real prepared statements (<code>ATTR_EMULATE_PREPARES=false</code>) in 100% of queries, including the LIKE search
4	Plain-text passwords	<code>password_hash()/password_verify()</code> (bcrypt); dummy-hash verification when the user doesn't exist (anti-timing)
5	Session fixation	<code>session_regenerate_id(true)</code> after every successful login

6	CSRF (cookie-based API)	Mandatory custom X-Requested-With header on POST/DELETE + SameSite=Lax + HttpOnly cookie
7	Login brute force	PER-USERNAME lockout persisted in the DB (5 attempts / 5 min) — clearing cookies no longer resets the counter; all time math uses MySQL's clock (see section 10, case 2)
8	Internal details leaking in errors	display_errors=0 + global handlers that answer a generic message and log details server-side only
9	XSS when displaying data	escapeHtml() on the frontend before inserting any text into the DOM (defense in depth)
10	Invalid input	Server-side validation of date, time range (start < end), hour bounds (0–24), client existence, description length
11	Misconfigured CORS	Eliminated by design: same origin for frontend and backend, zero CORS headers
12	Internal files served by Apache	.htaccess with Require all denied on database/, scripts/, backend/config and includes/ — before, seed_data.json (test passwords) answered 200 to anyone; confirmed 403 after

7. Business decisions (the two the exercise leaves open)

7.1 Overlapping hours on the same day (real example: August 6th)

Decision: don't block; detect and flag visually. The overlapping record is saved, but both the creation response and the listing flag it (a "⚠ traslape" badge) so the consultant or an admin can review it.

Why: a consultant can have legitimate reasons (fast context switching, a call that overlapped a meeting, a later correction). Blocking turns a *data-to-review* problem into a *broken-workflow* problem; silently trimming hours would be worse (the system would quietly decide which record is "the right one"). Flagging keeps a human in charge of the decision. The extension point for a stricter rule later already exists in the code.

7.2 Visibility of the financial summary between consultants

Decision: each consultant sees only their own summary (and only their own records); only the administrator sees the aggregate of everyone, or filters by consultant.

Why: billable hours are tied to what clients are charged and to each person's performance — sensitive information in the same sense as a salary, not a neutral operational fact. The exercise already sets the equivalent precedent for deletion ("only their own records"); extending the same ownership criterion to reading is consistent. The alternative (everyone sees everything) was considered and discarded: it adds nothing to a consultant's work and does create a privacy risk with no business justification.

8. Languages and tools: what was used and why

Language / tool	Role in the project	Why this one
PHP 8	The whole backend: API, sessions, MySQL connection	Runs natively on XAMPP's Apache with no build step or separate process — the requirement was "run it from XAMPP"
MySQL / SQL	Storage with real referential integrity	In this domain an orphaned record must be impossible (foreign keys), not merely "unlikely"
JavaScript (vanilla)	All frontend logic: fetch, session, rendering, forms	A 2-screen SPA doesn't justify a framework + bundler; it opens straight in the browser with no npm install
HTML5	Semantic structure of the single page	—
CSS3	Layout, glassmorphism and ALL the 3D animations	Native transform/perspective/@keyframes are enough without any library; only transform and opacity are animated (GPU) so it stays fluid; prefers-reduced-motion respected
Python 3	Independent summary verification (verify_summary.py) and the 15-test API suite	Deliberately a different language from the backend: if verification shared code with summary.php, one bug could

	(test_api.py, unittest, zero dependencies)	look right on both sides. Writing the math twice, in two languages, makes agreement real evidence
Bash / PowerShell	During development: curl/mysql testing. As a deliverable: check_all.sh/.ps1 (linting + both Python suites in one command)	The automation that ties PHP + JS + Python into one verifiable place, with a 0/1 exit code ready for pre-commit or CI
Apache (.htaccess)	Root DirectoryIndex; Require all denied on internal folders; no-cache on JS/CSS	Security and correctness at the layer where the problem lives (the server), not app-side patches
Git + GitHub	Incremental history (37+ commits) and delivery to the interviewer	See section 12

9. AI usage (Claude Code / Claude)

The project was built with **Claude Code** as a programming assistant in an interactive session. This is exactly what the exercise asks to document: what was requested, which decisions were made with it, and what had to be corrected from what it proposed.

9.1 What was requested

- Build the complete system described in the exercise, from scratch, in PHP + MySQL (XAMPP) + JS with 3D animations, all code commented in Spanish and English and organized into folders.
- Before writing code, the AI asked to confirm two ambiguities: whether to build the system or only deliver prompts, and what Python's role would be. Resolved: build everything, with Python as the independent verifier.
- **Actually test, not just write:** run the schema and seed against real MySQL, hit every endpoint with curl (including impersonation and unauthorized-access attempts), run the Python scripts against the live API, and take real browser screenshots (via Chrome DevTools Protocol) to visually confirm the UI.

9.2 Decisions made while working with the AI

- The two open business decisions (section 7): the AI proposed each one **with an explicit, verifiable justification**, not as an arbitrary rule — the acceptance criterion was never "the AI wrote it" but "the reasoning holds if someone challenges it".

- Not using the starter repository (the person's decision, not the AI's) and the PHP/MySQL/JS/Python stack (joint decision, confirmed at the start).
- What NOT to add: pagination for 20 demo rows, record editing nobody asked for, a light theme — documented because adding unjustified features is over-engineering that subtracts, not adds.

9.3 What had to be corrected from the AI's proposals (with evidence)

1. **Bar chart contrast:** the first version rendered three visually identical bars even though they measured 100%, 54% and 23%. Caught by comparing computed DOM widths against a real screenshot — the logic was fine, the CSS wasn't.
2. **Entry-point routing:** the first index.php draft include'd the frontend HTML (breaking every relative path); changed to a redirect computed from SCRIPT_NAME.
3. **The phpMyAdmin seed:** worked from the console, failed in real phpMyAdmin (#1701, TRUNCATE + foreign keys). See section 10, case 1.
4. **The login lockout never locked:** PHP's and MySQL's clocks were 8 hours apart on the same machine. See section 10, case 2.
5. **The monthly summary never refreshed itself,** and the first test of the fix was misleading due to browser caching. See section 10, case 5.

The pattern across the whole project: the AI proposes fast, but each piece was only accepted with real execution evidence (curl, screenshots, hex bytes, network traffic) — never because "the code looks right". Two of the most important bugs (the corrupted accents and the stale summary) were found by the person testing the app in their own browser, not by the AI or the automated tests: the final human review is not optional.

10. Real cases: bugs found and how they were fixed

Five cases that genuinely happened during development — each with root cause and fix, not just the symptom (full detail in docs/APRENDIZAJES.md §5):

#	Symptom	Root cause	Fix
1	phpMyAdmin failed with #1701 importing seed.sql (console worked fine)	phpMyAdmin's "enable foreign key checks" checkbox overrides the script's SET FOREIGN_KEY_CHECKS=0, and TRUNCATE fails on FK-referenced tables even when empty	Remove the need for TRUNCATE at the root (the script already starts from DROP DATABASE; tables are born empty)

2	The login logout never triggered, with no visible error	PHP and MySQL clocks ~8 hours apart (separately configured timezones); the window computed in PHP never matched MySQL's stored timestamps	All time math inside the SQL query (NOW()/INTERVAL/TIMESTAMPDIFF) — a single source of truth for "what time is it"
3	check_all.ps1 reported failure even with all 15 tests passing	PowerShell 5.1 wraps a native exe's stderr into ErrorRecords when redirecting with 2>&1, and unittest writes its results to stderr by design	Delegate redirection to cmd /c and read the real \$LASTEXITCODE
4	Accents rendered as " " on the real screen (found by the person, not the AI)	Correct UTF-8 bytes were reinterpreted under CP437 during a console import without forcing the encoding — permanent corruption in storage, verified with HEX()	SET NAMES utf8mb4 as seed.sql's first statement: the file declares its own encoding instead of assuming the client's
5	The monthly summary showed stale numbers after creating a record	Two stacked bugs: (a) loadSummary() only fired when the picker's value changed; (b) the first test of the fix was misleading because the browser had cached the pre-fix JS (Apache sent no Cache-Control on static files)	(a) refresh the visible panel on every tab switch and after create/delete; (b) frontend/.htaccess with no-cache, must-revalidate for .js/.css

11. Testing and verification

- **scripts/verify_summary.py** — the exercise asks not to blindly trust the summary number: this script recomputes, in Python, from seed_data.json, how many billable hours each client should report per month and per consultant, logs into the real backend, and compares. Result: all 17 cross-checks (2 months × consultants + the admin aggregate) match.
- **scripts/test_api.py** — 15 automated tests (unittest, zero dependencies) codifying everything that was tested by hand: authentication (401 without a session, 401 on a bad password, the hash never appears in responses), the exercise's point-4 impersonation, owner authorization on delete (403 foreign / 200 own / admin any), record and summary visibility between consultants, overlap detection with the August 6th example, SQL injection on search, and the login logout (using a made-up username so no real demo account gets locked). Every test cleans up its own data.
- **scripts/check_all.sh / .ps1** — one command: syntax of every PHP file (php -l), every JS file (node --check), test_api.py, and verify_summary.py. Exits 0 only if everything passed.

12. Git and GitHub: how they were used and what was learned

Git is not a feature of the program: the app runs identically without the `.git` folder. It is the tool that delivers **the construction process**, commit by commit — exactly what the exercise asks for ("normal incremental commits, not a single final commit").

- **37+ incremental commits**, each with a message that explains intent (not just the what); every commit builds and works on its own — no broken "WIP" commits mid-history.
- The order tells the real story: data first (schema/seed), then the shared security layer, then endpoints, frontend, Python verification, documentation — and then the improvement rounds with their found-and-fixed bugs.
- The full per-commit table, with an English translation of each commit's purpose, lives in `docs/GIT.md`.

Git vs. GitHub: git runs locally; GitHub is the service that hosts the repository so it can be shared. Publishing it (exact steps in `docs/GIT.md`): create the repository on github.com without an initial README, `git remote add origin ...`, `git branch -M main`, `git push -u origin main`, confirm the Commits tab shows the history, and share the URL with the interviewer (or invite them as a collaborator if private). What the reviewer sees: the full code, the README on the front page, and every commit with its diff — the process, not just the result.

13. Final learnings and interview checklist

- **Authentication and authorization are two different questions** — and the code separates them as two different functions.
- **Never trust client-controlled data** to decide identity or ownership — the server session is the only authority.
- **"Works in my test" ≠ "works in the real tool"** (phpMyAdmin, PowerShell, a browser with cache).
- **When two systems must agree on something (time, encoding), make one of them the owner** — don't synchronize two.
- **The hardest bugs fail silently** — the lockout that never locked threw no exception at all.
- **Automated tests reduce risk but don't replace opening the real app** — two bugs were found by the person, not the machine.
- **Restraint is a feature:** knowing what NOT to add (and being able to say why) is worth as much as what was added.

If they ask...	The answer lives in...
----------------	------------------------

Why didn't you use the starter repo they gave you?	NOTES.md (top) — explicit request not to copy-paste; everything written from scratch
How do you stop one consultant seeing/deleting another's data?	NOTES.md §1.1–1.2 + backend/includes/auth.php
How do you know the summary is correct?	scripts/verify_summary.py + NOTES.md §4 (real evidence)
What happens with overlapping hours?	NOTES.md §2.1 — a justified decision
Why Python if the backend is PHP?	Section 8 — genuinely independent verification
Tell me about a real bug you solved	Section 10 — five of them, with root causes
Do you have automated tests?	test_api.py (15) + check_all — section 11
Why so many small commits?	docs/GIT.md in full — section 12